

Laporan Praktikum Kontrol Cerdas

Nama : Alam Syah Putra Aulia
NIM : 224308003
Kelas : TKA 6A
Akun Github (Tautan) : <https://github.com/Alamsyah003>
Student Lab Assistant : Yulia Brillianty

1. Judul Percobaan

Canny Edge Detection & Lane Detection with Instance Segmentation

2. Tujuan Percobaan

Tujuan dari praktikum ini adalah sebagai berikut

1. Mahasiswa dapat memahami Konsep dasar canny edge detection sebagai metode dasar deteksi tepi.
2. Mahasiswa dapat mengetahui cara menggunakan instance segmentation untuk deteksi jalur rel kereta (Lane Detection).
3. Mahasiswa dapat mengetahui bagaimana cara menggabungkan metode canny edge detection dengan instance segmentation untuk meningkatkan deteksi jalur.

3. Landasan Teori

Canny Edge Detection adalah salah satu metode deteksi tepi yang paling banyak digunakan karena keandalannya dan fleksibilitasnya. Algoritma ini dikembangkan oleh John F. Canny dengan tujuan mendeteksi tepi gambar secara optimal dengan meminimalkan gangguan akibat *noise*. Prosesnya terdiri dari tiga tahap utama untuk mengekstraksi tepi gambar secara efektif. Sebagai langkah awal, dilakukan *preprocessing* dengan menerapkan efek kabur untuk mengurangi noise sebelum melanjutkan ke tahap berikutnya.

Lane detection adalah proses mengenali jalur atau marka jalan dalam gambar atau video, yang digunakan dalam sistem bantuan pengemudi (ADAS) dan kendaraan otonom. Sementara itu,

instance segmentation merupakan teknik dalam visi komputer yang tidak hanya mengklasifikasikan objek, tetapi juga menentukan batas piksel untuk setiap objek secara individual. Dalam konteks ini, *instance segmentation* berfungsi untuk mendeteksi dan membedakan setiap jalur secara terpisah, bukan sekadar mendeteksi tepi jalur secara keseluruhan. Model *deep learning* seperti *Mask R-CNN* atau *DeepLab* digunakan untuk melakukan segmentasi dan klasifikasi marka jalan pada gambar atau video.

4. Analisis dan Diskusi

Analisis

Pada percobaan minggu ke-6 ini, saya melakukan eksperimen dalam mendeteksi jalur rel kereta menggunakan dua metode, yaitu *Canny Edge Detection* dan *Lane Detection* dengan *Instance Segmentation*. Dalam penelitian ini, saya memanfaatkan YOLOv8-seg sebagai model *instance segmentation* serta beberapa *library* pendukung seperti NumPy dan lainnya.

Proses pembuatan program dibagi menjadi tiga bagian. Program pertama berfungsi untuk mendeteksi tepi menggunakan metode *Canny Edge Detection*, menghasilkan output berupa gambar rel di mana area jalur rel tampak berwarna putih, sementara area lainnya berwarna hitam. Program kedua menggunakan metode *Lane Detection* dengan *Instance Segmentation* (YOLOv8-seg), menghasilkan output berupa gambar dengan jalur rel yang ditandai menggunakan segmentasi warna. Program ketiga mengombinasikan kedua metode tersebut, menghasilkan output berupa gambar objek dengan *bounding box* dan label, sekaligus menampilkan struktur tepi dari objek yang terdeteksi.

Hasil percobaan menunjukkan bahwa metode *instance segmentation* lebih unggul dalam mendeteksi jalur rel dibandingkan dengan *Canny Edge Detection*, namun memerlukan daya komputasi yang lebih besar. Kombinasi kedua metode ini mampu meningkatkan akurasi dalam deteksi jalur serta menghasilkan deteksi yang lebih stabil. Selain itu, pengaturan parameter *threshold* pada *Canny Edge Detection* berpengaruh terhadap hasil deteksi tepi objek. Jika *threshold* terlalu rendah, banyak noise dari tekstur rel atau bayangan yang turut terdeteksi sebagai tepi. Sebaliknya, jika *threshold* terlalu tinggi, hanya bagian jalur yang memiliki kontras tinggi yang akan terdeteksi, sementara jalur rel yang pudar atau tertutup sebagian bisa tidak terdeteksi dalam hasil akhir.

Diskusi

- Pemilihan antara *Canny Edge Detection* dan *Instance Segmentation* bergantung pada kondisi gambar, tujuan deteksi, serta kebutuhan komputasi. Jika dibutuhkan deteksi yang

cepat dan ringan, hanya untuk mengenali struktur atau kontur suatu objek, serta gambar memiliki kontras tinggi dan tidak terlalu kompleks, maka *Canny Edge Detection* menjadi pilihan yang lebih tepat.

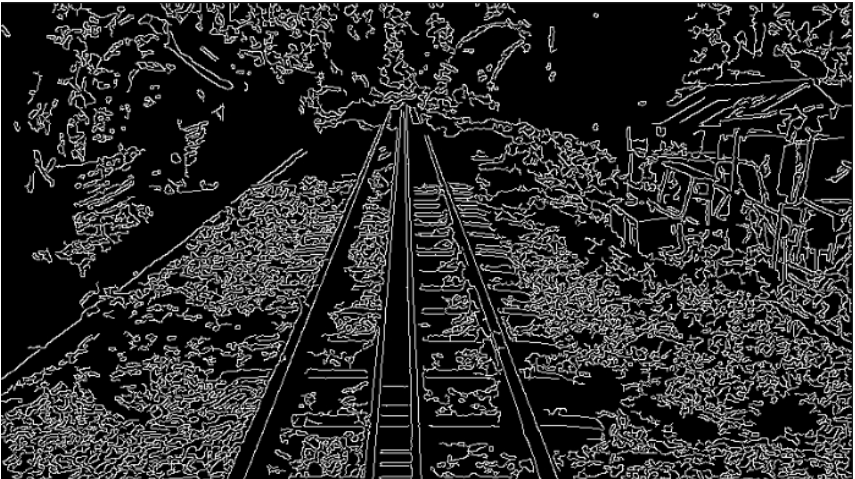
- Agar YOLOv8-seg dapat mendeteksi jalur dengan lebih akurat, diperlukan *tuning* parameter baik pada tahap pelatihan (*training*) maupun saat inferensi (prediksi). Pada tahap pelatihan, hal ini dapat dilakukan dengan menggunakan dataset yang beragam dan berkualitas, menerapkan augmentasi data, serta menyesuaikan hyperparameter agar model lebih stabil. Sementara itu, pada tahap inferensi, tuning parameter dapat dilakukan dengan menyesuaikan nilai *confidence* dan *IoU threshold* untuk meningkatkan ketepatan deteksi. Selain itu, hasil deteksi juga dapat diperbaiki melalui *post-processing*, seperti menerapkan filter area atau mengombinasikannya dengan *Canny Edge Detection* agar menghasilkan deteksi yang lebih optimal.
- Metode *Canny Edge Detection* dan YOLOv8-seg dapat diterapkan dalam sistem navigasi kereta otomatis dengan menentukan peran masing-masing dalam sistem tersebut. Pertama, perlu dirancang arsitektur sistem navigasi yang memanfaatkan deteksi jalur, di mana YOLOv8-seg digunakan untuk mengenali jalur rel, sementara *Canny Edge Detection* berfungsi mengekstraksi tepi jalur. Dengan kombinasi kedua metode ini, sistem dapat menghasilkan jalur yang dapat diikuti oleh kereta otomatis, memungkinkan sistem navigasi untuk mengontrol pergerakan kereta secara optimal.

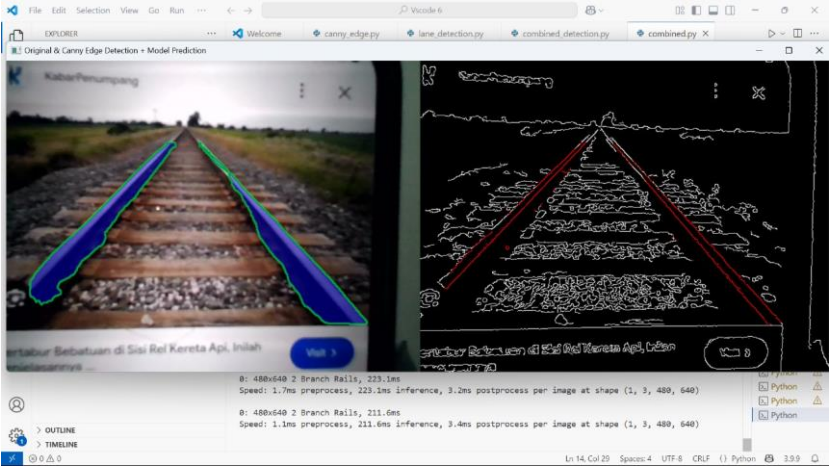
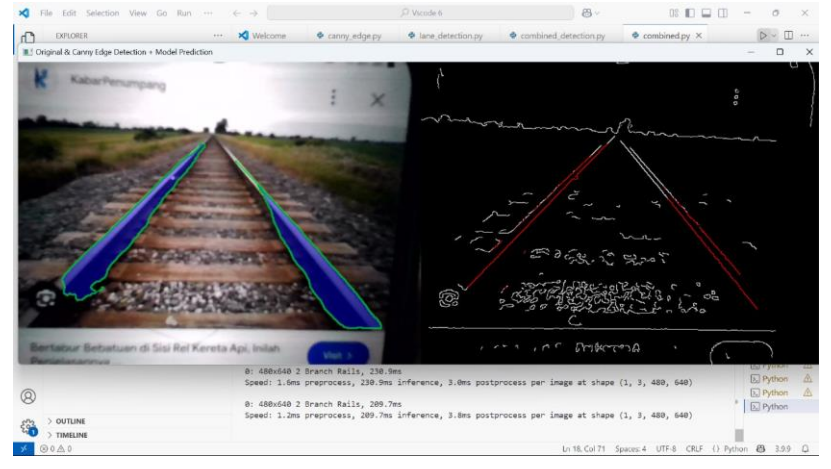
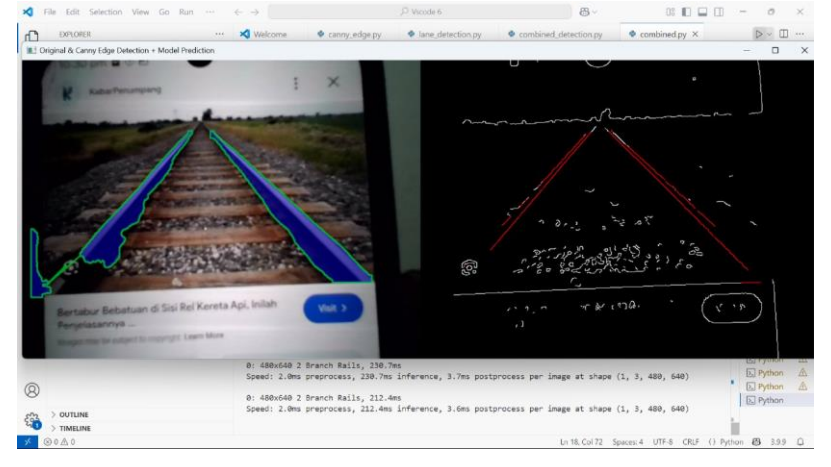
5. Assignment

Pada assignment ini memanfaatkan OpenCV, NumPy, Torch, dan YOLO dari Ultralytics untuk melakukan deteksi objek dan segmentasi secara real-time melalui kamera. Model YOLO dimuat dari file `best.pt`, dan jika terjadi kesalahan, program akan menampilkan pesan error dan keluar. Setelah berhasil mengakses kamera, program membaca frame secara berulang, mengonversinya ke *grayscale* untuk diterapkan *Canny Edge Detection*, serta ke format RGB untuk proses deteksi oleh YOLO. Jika terdeteksi objek, program menampilkan *bounding box* atau *mask* berwarna biru transparan dengan garis hijau di sekeliling objek. Hasil deteksi dari YOLO digabungkan dengan hasil tepi dari Canny, di mana tepi objek berwarna merah dan tepi lainnya tetap putih. Tampilan akhir disajikan dalam satu jendela, dengan frame asli di sisi kiri dan hasil deteksi tepi di sisi kanan. Program akan berhenti saat pengguna menekan tombol 'q'. Meskipun fungsionalitasnya sudah cukup baik, peningkatan masih dapat dilakukan, seperti penanganan pemilihan kamera yang lebih fleksibel, peningkatan efisiensi deteksi, atau penyempurnaan visualisasi overlay.

6. Data dan Output Hasil Pengamatan

Sajikan data dan hasil yang diperoleh selama percobaan. Gunakan tabel untuk menyajikan data jika diperlukan.

No	Variabel	Hasil Pengamatan
1	Foto Asli	
2	Deteksi Rel Metode Canny Edge Detection	
3	Deteksi Rel Metode Lane Detection	

4	Deteksi Rel Metode Canny Edge Detection dan Lane Detection	
5	low_threshold=50, high_threshold=150	
6	low_threshold=150, high_threshold=250	
7	low_threshold=200, high_threshold=300	

7. Kesimpulan

Dari percobaan yang dilakukan, deteksi tepi rel menggunakan metode *Canny Edge Detection* dan *Lane Detection* dengan *Instance Segmentation* (YOLOv8-seg) telah diuji melalui tiga program. Program pertama mendeteksi tepi rel dengan *Canny Edge Detection*, program kedua menggunakan YOLOv8-seg, dan program ketiga mengombinasikan keduanya. Hasilnya menunjukkan bahwa *Instance Segmentation* lebih akurat dalam mendeteksi tepi rel dibandingkan *Canny Edge Detection*, tetapi membutuhkan daya komputasi lebih besar. Namun, dengan menggabungkan kedua metode, akurasi deteksi jalur meningkat dan hasilnya lebih stabil.

8. Saran

Berdasarkan hasil percobaan, beberapa saran untuk meningkatkan akurasi dan performa deteksi tepi rel adalah menambah variasi dataset serta melakukan *tuning* parameter yang tepat. Selain itu, program sebaiknya diimplementasikan pada video atau *real-time processing* untuk pengujian yang lebih optimal.

9. Daftar Pustaka

- Abdul Muthalib, M., Irfan, I., Kartika, K., & Selamat Meliala, S. M. (2023). PENGIRAAN POSE MODEL MANUSIA PADA REPETISI KEBUGARAN AI PEMOGRAMAN PYTHON BERBASIS KOMPUTERISASI. *INFOTECH journal*, 9(1), 11-19.
<https://doi.org/10.31949/infotech.v9i1.4233>
- Rizki, A. B., & Zuliarso, E. (2020). *Klasifikasi Teknik Bulutangkis Berdasarkan Pose Dengan Convolutional Neural Network*. 10(02).